

School Management App

Service Réteg – Fejlesztői dokumentáció

Laravel 12 + Clean Code elvek | 2026

1. Mi a Service réteg és miért kell?

A Service réteg a Clean Code architektúra egyik alapköve Laravel projektekben. Célja az üzleti logika elkülönítése a Controller rétegtől, hogy minden osztálynak egyetlen, jól meghatározott felelőssége legyen.

Ezt a tervezési elvet Single Responsibility Principle-nek (SRP) nevezzük – ez a SOLID elvek egyike.

1.1 A probléma – Service réteg nélkül

Service réteg nélkül a Controller túl sok mindent csinál egyszerre: validál, üzleti logikát futtat, adatbázist kezel és választ formáz. Ez nehezen tesztelhető, nehezen karbantartható kódhoz vezet.

```
// ROSSZ megközelítés - Controller túl sok mindent csinál
class StudentController extends Controller
{
    public function store(Request $request)
    {
        // 1. Validáció
        $request->validate(['name' => 'required', ...]);

        // 2. Üzleti logika - roll_number generálás
        $rollNumber = 'STU-' . strtoupper(uniqid());

        // 3. Adatbázis művelet
        $student = Student::create([...$request->all(), 'roll_number' =>
$rollNumber]);

        // 4. Email küldés
        Mail::to($student->email)->send(new WelcomeMail($student));

        // 5. Válasz formázás
        return response()->json($student);
    }
}
```

1.2 A megoldás – Service réteggel

A Service réteggel minden osztály csak a saját felelősségével foglalkozik. A Controller csak fogadja a kérést és visszaadja a választ. Az összes üzleti logika a Service-ben van.

```
// JÓ megközelítés - mindenki a saját dolgával foglalkozik

// Controller - csak delegál
class StudentController extends Controller
{
    public function store(StoreStudentRequest $request)
    {
        $student = $this->studentService->createStudent($request->validated());
        return new StudentResource($student);
    }
}

// Service - itt van az üzleti logika
class StudentService
{
    public function createStudent(array $data): Student
    {
        $data['roll_number'] = $this->generateRollNumber();
        $student = Student::create($data);
        Mail::to($student->email)->send(new WelcomeMail($student));
        return $student;
    }
}
```

1.3 A rétegek felelősségei

Réteg	Felelőssége	Nem feladata
Form Request	Validáció, jogosultság ellenőrzés	Üzleti logika, adatbázis
Controller	Kérés fogadása, Service hívás, válasz	Validáció, üzleti logika
Service	Üzleti logika, számítások, tranzakciók	HTTP kérés/válasz kezelés
Model	Adatbázis kapcsolat, kapcsolatok	Üzleti logika
API Resource	JSON válasz formázása	Adatbázis, logika

2. Dependency Injection

A Dependency Injection (DI) egy tervezési minta, amellyel a Controller nem maga hozza létre a Service példányt, hanem a Laravel konténer automatikusan "befecskendezi" azt.

2.1 Tight Coupling vs Loose Coupling

```
// ROSSZ - Tight Coupling (szoros kapcsolat)
class StudentController extends Controller
{
    public function index()
    {
        $service = new StudentService(); // Controller függ a Service-től
        return $service->getAllStudents();
        // Ha a StudentService konstruktora megváltozik → ez elromlik
    }
}
```

```
// JÓ - Loose Coupling (laza kapcsolat)
class StudentController extends Controller
{
    public function __construct(
        private StudentService $studentService
    ) {}
    // A Laravel IoC konténer automatikusan létrehozza
    // és átadja a StudentService példányt

    public function index()
    {
        return $this->studentService->getAllStudents();
    }
}
```

2.2 Miért jobb a Dependency Injection?

Tesztelhetőség – tesztelésnél kicserélhető egy "mock" (fake) Service-re
Rugalmasság – ha a Service megváltozik, a Controller nem változik
Egyértelműség – a konstruktorból látszik mire van szüksége az osztálynak
Laravel konténer – a framework kezeli az objektumok életciklusát

3. StudentService

A StudentService a diákokkal kapcsolatos összes üzleti logikát tartalmazza. Ez a legteljesebb Service a projektben, mintaként szolgál a többihez.

3.1 getAllStudents() – lapozással

```
public function getAllStudents(int $perPage = 15): LengthAwarePaginator
{
    return Student::with(['schoolClass'])
        ->orderBy('name')
        ->paginate($perPage);
}
```

Miért paginate() és nem all()?

Ha 1000 diák van az adatbázisban, az összes egyszerre való lekérdezése lassú és memóriagényes lenne. A paginate() csak az aktuális oldal rekordjait kéri le az adatbázisból – alapértelmezetten 15-öt oldalanként. A LengthAwarePaginator automatikusan tartalmazza az összes lapozási információt (total, per_page, current_page, last_page).

3.2 getStudentById() – eager loadinggal

```
public function getStudentById(int $id): Student
{
    return Student::with([
        'schoolClass',
        'guardians',
        'attendances',
        'examResults',
        'fees',
    ])->findOrFail($id);
}
```

Miért findOrFail() és nem find()?

A find() null-t ad vissza ha nem találja a rekordot – ezt kézzel kéne ellenőrizni. A findOrFail() automatikusan 404-es HTTP hibát dob ha nem létezik az id, így a Controller nem kell hogy foglalkozzon ezzel az esettel. Kevesebb kód, ugyanolyan biztonság.

3.3 createStudent() – roll_number generálással

```
public function createStudent(array $data): Student
{
    $data['roll_number'] = $this->generateRollNumber();
    return Student::create($data);
}
```

```
private function generateRollNumber(): string
{
    $year    = now()->year;
    $count   = Student::withTrashed()->count() + 1;
    $number  = str_pad($count, 5, '0', STR_PAD_LEFT);
    return "STU-{$year}-{$number}";
    // Példa: STU-2026-00001
}
```

Miért withTrashed() a count()-nál?

Ha törölt diákok is vannak az adatbázisban (soft delete), azokat is bele kell számolni a sorszámba – különben előfordulhatna két azonos roll_number. A withTrashed() a soft delete-elt rekordokat is beleszámolja. A metódus private, mert csak a Service-en belül van rá szükség.

3.4 updateStudent() – fresh() használatával

```
public function updateStudent(Student $student, array $data): Student
{
    $student->update($data);
    return $student->fresh();
    // fresh() újra lekérdezi az adatbázisból a frissített adatokat
}
```

Miért fresh() és nem return \$student?

Az update() módosítja az adatbázisban a rekordot, de a PHP memóriában lévő \$student objektum még a régi adatokat tartalmazza. A fresh() egy új lekérdezéssel frissen kéri le az objektumot az adatbázisból, így mindig naprakész adatot adunk vissza a frontendnek.

3.5 deleteStudent() – soft delete

```
public function deleteStudent(Student $student): void
{
    $student->delete();
    // Nem fizikai törlés! A SoftDeletes trait miatt
    // csak a deleted_at mező kap értéket.
    // Az adat megmarad és visszaállítható: $student->restore();
}
```

4. TeacherService

A TeacherService a tanárokkal kapcsolatos logikát kezeli. Érdekesség az onDelete('set null') viselkedése: ha egy tanárt törölünk, az osztályok és tantárgyak megmaradnak, csak az osztályfőnök/tanár mező lesz null.

4.1 getTeacherById() – beágyazott eager loading

```
public function getTeacherById(int $id): Teacher
{
    return Teacher::with([
        'subjects',
        'classes',
        'timetables.subject',    // beágyazott: timetable + subject
        'timetables.schoolClass',
    ])->findOrFail($id);
}
```

Mit jelent a 'timetables.subject'?

Ez beágyazott (nested) eager loading. Először betölti az összes timetable bejegyzést a tanárhoz, majd minden timetable-höz betölti a kapcsolódó subject-et is. Mindez összesen 3 adatbázis lekérdezéssel történik függetlenül attól hány órarendbejegyzése van a tanárnak.

4.2 getTeacherScheduleByDay() – napi beosztás

```
public function getTeacherScheduleByDay(Teacher $teacher, string $day):
Collection
{
    return $teacher->timetables()
        ->with(['subject', 'schoolClass'])
        ->where('day', $day)
        ->orderBy('start_time')
        ->get();
}
// Használat: $service->getTeacherScheduleByDay($teacher, 'monday');
```

5. AttendanceService

Az AttendanceService a legösszetettebb Service a projektben. Két fontos koncepciót alkalmaz: az adatbázis tranzakciókat és a statisztikai számításokat.

5.1 DB::transaction() – adatbázis tranzakció

```

public function recordClassAttendance(
    SchoolClass $schoolClass,
    string $date,
    array $attendanceData
): Collection {
    return DB::transaction(function () use ($schoolClass, $date,
    $attendanceData) {

        // Töröljük az aznapi meglévő bejegyzéseket (újrarögzítés esetén)
        Attendance::where('class_id', $schoolClass->id)
            ->where('date', $date)
            ->whereNotNull('student_id')
            ->delete();

        // Minden diákhoz létrehozunk egy jelenléti bejegyzést
        $records = collect($attendanceData)->map(function ($item) use
        ($schoolClass, $date) {
            return Attendance::create([
                'student_id' => $item['student_id'],
                'class_id'   => $schoolClass->id,
                'date'       => $date,
                'status'     => $item['status'],
                'note'       => $item['note'] ?? null,
            ]);
        });

        return $records;
    });
}

```

Miért kell a DB::transaction()?

Képzeld el hogy 30 diák jelenlétét rögzítjük. Ha a 15. diáknál hiba történik, az első 14 már el van mentve – az adatbázis inkonzisztens állapotba kerül. A transaction() garantálja hogy VAGY minden elmenti (commit), VAGY ha hiba van, SEMMI sem mentődik el (rollback). Tranzakciót mindig akkor használunk amikor több adatbázis műveletet kell "egységként" kezelni.

5.2 getStudentAttendanceStats() – statisztika számítás

```

public function getStudentAttendanceStats(
    Student $student,
    ?string $from = null,
    ?string $to   = null
): array {
    $query = $student->attendances();

    if ($from) { $query->where('date', '>=', $from); }
    if ($to)   { $query->where('date', '<=', $to); }
}

```

```

// Egyetlen lekérdezéssel megkapjuk az összes státusz darabszámát
$status = $query->selectRaw('status, COUNT(*) as count')
    ->groupBy('status')
    ->pluck('count', 'status')
    ->toArray();

$present = $stats['present'] ?? 0;
$absent  = $stats['absent']  ?? 0;
$late    = $stats['late']    ?? 0;
$excused = $stats['excused'] ?? 0;
$total   = $present + $absent + $late + $excused;

// Késés félig jelenlétnek számít (üzleti döntés)
$percentage = $total > 0
    ? round(($present + $late) / $total) * 100, 2)
    : 0;

return [
    'total'      => $total,
    'present'    => $present,
    'absent'     => $absent,
    'late'       => $late,
    'excused'    => $excused,
    'percentage' => $percentage,
];
}

```

Miért selectRaw() és groupBy()?

A selectRaw('status, COUNT(*) as count') + groupBy('status') egyetlen SQL lekérdezéssel adja vissza az összes státusz darabszámát. Ez sokkal hatékonyabb mint külön lekérdezni a present, absent, late és excused darabszámokat. A ?? 0 operátor az alapértelmezett értéket adja meg arra az esetre ha egy státusból nincs egyetlen bejegyzés sem.

6. ExamService

Az ExamService kezeli a vizsgákat és az eredmények rögzítését. Tartalmaz automatikus osztályzat számítást és statisztika generálást.

6.1 recordResults() – tömeges eredményrögzítés

```

public function recordResults(Exam $exam, array $resultsData): void
{
    DB::transaction(function () use ($exam, $resultsData) {
        foreach ($resultsData as $resultData) {

            // Automatikus osztályzat és státusz számítás

```



```

        $grade = $this->calculateGrade($resultData['marks_obtained'],
        $exam->total_marks);
        $status = $resultData['marks_obtained'] >= $exam-
        >passing_marks ? 'pass' : 'fail';

        // Ha már van eredmény → frissítés, ha nincs → létrehozás
        ExamResult::updateOrCreate(
            ['exam_id' => $exam->id, 'student_id' =>
            $resultData['student_id']],
            ['marks_obtained' => $resultData['marks_obtained'],
            'grade' => $grade, 'status' => $status,
            'remarks' => $resultData['remarks'] ?? null]
        );
    }
});
}

```

Miért updateOrCreate()?

Az updateOrCreate() két dolgot csinál egyszerre: ha létezik már eredmény ehhez a diákhoz ezen a vizsgán, frissíti azt. Ha nem létezik, létrehozza. Ez megakadályozza a duplikált rekordok keletkezését és lehetővé teszi az eredmények újrarögzítését javítás esetén.

6.2 calculateGrade() – osztályzat számítás

```

private function calculateGrade(int $marksObtained, int $totalMarks):
string
{
    $percentage = ($marksObtained / $totalMarks) * 100;

    return match(true) {
        $percentage >= 90 => 'A+',
        $percentage >= 80 => 'A',
        $percentage >= 70 => 'B+',
        $percentage >= 60 => 'B',
        $percentage >= 50 => 'C+',
        $percentage >= 40 => 'C',
        $percentage >= 33 => 'D',
        default           => 'F',
    };
}

```

Miért match() és nem if-else?

A match() kifejezés PHP 8.0-ban jelent meg. Ugyanazt csinálja mint az if-else lánc, de sokkal tömörebb és olvashatóbb. A match(true) azt jelenti hogy az első igaz feltételnél megáll és visszaadja az értéket. A private kulcsszó azért van mert ez belső segédfüggvény – kívülről nem hívható.

6.3 getExamStatistics() – vizsga statisztika

```
public function getExamStatistics(Exam $exam): array
{
    $results = $exam->results;

    if ($results->isEmpty()) {
        return ['total_students'=>0, 'average'=>0, ...];
    }

    $passCount = $results->where('status', 'pass')->count();
    $total      = $results->count();

    return [
        'total_students' => $total,
        'average'        => round($results->avg('marks_obtained'), 2),
        'highest'        => $results->max('marks_obtained'),
        'lowest'         => $results->min('marks_obtained'),
        'pass_count'     => $passCount,
        'fail_count'     => $results->where('status', 'fail')->count(),
        'pass_rate'      => round(($passCount / $total) * 100, 2),
    ];
}
```

7. FeeService

A FeeService a díjak és fizetések kezelését végzi. Tartalmaz automatikus bizonylat generálást és lejárt díjak tömeges frissítését.

7.1 markAsPaid() – befizetés rögzítése

```
public function markAsPaid(Fee $fee): Fee
{
    $fee->update([
        'status'          => 'paid',
        'paid_date'       => now()->toDateString(),
        'receipt_number' => $this->generateReceiptNumber(),
    ]);
    return $fee->fresh();
}

private function generateReceiptNumber(): string
{
    $year    = now()->year;
    $count   = Fee::whereYear('created_at', $year)->count() + 1;
    $number  = str_pad($count, 6, '0', STR_PAD_LEFT);
}
```

```

return "RCP-{$year}-{$number}";
// Példa: RCP-2026-000001
}

```

Miért külön markAsPaid() metódus?

A befizetés rögzítése nem egyszerű update – több mezőt kell egyszerre frissíteni (status, paid_date, receipt_number) és a bizonylatszámot automatikusan kell generálni. Ha ezt a Controllerbe tennénk, minden befizetési végpontnál újra kellene írni. A Service-ben egyszer van megírva és bárhonnán hívható.

7.2 updateOverdueStatuses() – tömeges frissítés

```

public function updateOverdueStatuses(): int
{
    return Fee::where('status', 'pending')
        ->where('due_date', '<', now())
        ->update(['status' => 'overdue']);
    // Visszaadja hány rekord lett frissítve
}

// Ezt egy Laravel Scheduled Command hívja naponta egyszer:
// $schedule->call(fn() => app(FeeService::class)-
>updateOverdueStatuses())
// ->daily();

```

Miért tömeges update() és nem egyenként?

A tömeges update() egyetlen SQL UPDATE lekérdezéssel frissíti az összes lejárt díjat. Ha egyenként tennénk (foreach + \$fee->update()), minden egyes rekordhoz egy külön SQL lekérdezés futna le. 100 lejárt díj esetén ez 100x lassabb lenne. A tömeges update mindig gyorsabb nagy adatmennyiségnél.

8. LibraryService

A LibraryService a könyvtári kölcsönzések kezelését végzi. Két atomi műveletet tartalmaz – könyv kölcsönzése és visszahozása – mindkettő tranzakcióban fut.

8.1 issueBook() – könyv kölcsönzése

```

public function issueBook(LibraryBook $book, array $data): LibraryIssue
{
    if ($book->available_copies <= 0) {

```

```

        throw new \Exception('Nincs elérhető példány.');
```

```

    }

    return DB::transaction(function () use ($book, $data) {
        $issue = LibraryIssue::create([
            'book_id'      => $book->id,
            'student_id' => $data['student_id'] ?? null,
            'teacher_id' => $data['teacher_id'] ?? null,
            'issue_date' => now()->toDateString(),
            'due_date'    => $data['due_date'],
            'status'      => 'issued',
            'fine'        => 0,
        ]);

        $book->decrement('available_copies');
        return $issue;
    });
}

```

Miért tranzakció a kölcsönzésnél?

Két dolgot kell egyszerre csinálni: létrehozni a kölcsönzési bejegyzést ÉS csökkenteni az elérhető példányok számát. Ha a bejegyzés létrejön de a decrement() hibára fut, az adatbázisban azt mutatja hogy ki van kölcsönözve de a szám nem csökkent – ez inkonzisztens állapot. A tranzakció garantálja hogy mindkettő sikerül vagy egyik sem.

8.2 returnBook() – könyv visszahozása és bírság számítás

```

public function returnBook(LibraryIssue $issue): LibraryIssue
{
    return DB::transaction(function () use ($issue) {
        $fine = $this->calculateFine($issue);

        $issue->update([
            'return_date' => now()->toDateString(),
            'status'      => 'returned',
            'fine'        => $fine,
        ]);

        $issue->book->increment('available_copies');
        return $issue->fresh();
    });
}

private function calculateFine(LibraryIssue $issue): float
{
    $today    = now()->startOfDay();
    $dueDate = Carbon::parse($issue->due_date)->startOfDay();

    if ($today <= $dueDate) { return 0; }
}

```

```
$daysLate = $today->diffInDays($dueDate);
return $daysLate * 50; // napi 50 Ft késedelmi díj
}
```

Miért Carbon::parse() és nem ->copy()?

A Carbon::parse() teljesen új Carbon objektumot hoz létre a dátumból – legyen az akár string akár Carbon objektum. Ez biztonságosabb mint a ->copy()->startOfDay(), mert a startOfDay() módosítja (mutálja) az eredeti objektumot. A Carbon::parse() esetén az eredeti \$issue->due_date változatlan marad.

9. NotificationService

A NotificationService az értesítések létrehozását és olvasottságuk kezelését végzi.

9.1 createNotification() – with() a Controllerből

```
public function createNotification(array $data, User $creator):
Notification
{
    return Notification::create([
        ...$data,          // spread operátor - az összes mezőt
        'created_by' => $creator->id,
        'sent_at'    => now(),
    ]);
}
```

Miért kapja a User objektumot és nem csak az id-t?

A Service aláírása egyértelműbb ha User objektumot vár – a hívó félnek nem kell tudnia hogy a Service a created_by mezőt állítja be. Az id kinyerése (\$creator->id) a Service feladata. Ez is a loose coupling elvét követi.

9.2 markAsRead() – firstOrCreate()

```
public function markAsRead(Notification $notification, User $user): void
{
    NotificationRead::firstOrCreate(
        ['notification_id' => $notification->id, 'user_id' => $user->id],
        ['read_at' => now()]
    );
}
```

```
}
```

Mit csinál a `firstOrCreate()`?

Az első tömbben lévő feltételekkel megkeresi a rekordot. Ha megtalálja, visszaadja azt – nem hoz létre újat. Ha nem találja, létrehozza a két tömb egyesítésével. Ez megakadályozza hogy valaki véletlenül kétszer is 'olvassnak' jelöljön egy értesítést.

9.3 `getUnreadCount()` – `whereDoesntHave()`

```
public function getUnreadCount(User $user): int
{
    return Notification::whereDoesntHave('reads', function ($query) use
($user) {
        $query->where('user_id', $user->id);
    })->count();
}

// A 'reads' az eager loading neve, nem a tábla neve!
// Notification modellben: public function reads(): HasMany { ... }
```

Miért `whereDoesntHave()` és nem `whereHas()`?

A `whereHas()` azokat adja vissza ahol LÉTEZIK a kapcsolódó rekord. A `whereDoesntHave()` ennek ellenkezője – azokat ahol NEM létezik. Olvasatlan értesítések = azok ahol NEM létezik `NotificationRead` bejegyzés az adott felhasználóhoz. Fontos: a 'reads' itt a `Notification` modellben definiált `reads()` metódus neve, nem a tábla neve!

10. Összefoglaló – Kulcsfogalmak

Fogalom	Mit jelent	Hol használtuk
Single Responsibility	Egy osztálynak egy felelőssége van	Minden Service osztály
Dependency Injection	Laravel konténer adja át a Service-t	Controller konstruktor
<code>paginate()</code>	Lapozva kéri le az adatokat	Minden listázó metódus
<code>findOrFail()</code>	404-et dob ha nem találja	Minden <code>getById()</code> metódus
<code>fresh()</code>	Újra lekérdezi az objektumot DB-ből	Minden <code>update()</code> metódus
<code>DB::transaction()</code>	Atomi művelet – vagy mind vagy semmi	Attendance, Library, Exam

updateOrCreate()	Update ha létezik, create ha nem	ExamService
withTrashed()	Soft delete-elt rekordokat is látja	generateRollNumber()
firstOrCreate()	Keresi, ha nincs létrehozza	NotificationService
whereDoesntHave()	Ahol NEM létezik kapcsolódó rekord	getUnreadCount()
Carbon::parse()	Biztonságos dátum objektum létrehozás	LibraryService
match()	PHP 8 tömör if-else kiváltó	calculateGrade()
selectRaw()+groupBy()	Egyetlen SQL-lel több számlálás	getAttendanceStats()
decrement/increment()	Atomi számlálás adatbázisban	LibraryService

Services mappa struktúra

```

app/
└── Services/
    ├── StudentService.php    ← CRUD + roll_number generálás
    ├── TeacherService.php    ← CRUD + napi beosztás
    ├── SchoolClassService.php ← CRUD + withCount
    ├── AttendanceService.php ← transaction + statisztika
    ├── ExamService.php       ← osztályzat számítás + bulk insert
    ├── FeeService.php        ← bizonylat generálás + tömeges update
    ├── LibraryService.php    ← kölcsönzés + bírság számítás
    └── NotificationService.php ← olvasottság + whereDoesntHave

```

– School Management App fejlesztési dokumentáció –